

目录

一、引言

- 1.1 编写目的
- 1.2 软件概述
- 1.3 运行环境
- 1.4 术语与缩略语

二、软件总体设计

- 2.1 软件需求概括
- 2.2 总体架构设计
- 2.3 模块划分与关系
- 2.4 场景系统设计

三、核心模块详细设计

- 3.1 游戏入口模块
- 3.2 场景管理模块
- 3.3 捞水果关卡玩法模块
- 3.4 每日限定玩法模块
- 3.5 切水果无尽玩法模块
- 3.6 图鉴、扭蛋与排行榜模块
- 3.7 云同步与数据上报模块

四、数据结构设计

五、数据接口设计

六、出错处理设计

七、性能优化设计

一、引言

1.1 编写目的

编写本设计说明书是软件开发过程的重要组成部分。本文档旨在详细描述深圳幸运呱科技有限公司别捞水果小游戏软件（以下简称“本软件”）的软件架构设计、核心模块设计、数据结构设计、接口设计及出错处理设计，为软件著作权登记提供技术性文档依据。

本文档面向软件著作权审查人员，全面展示本软件的技术架构、设计思路和实现方案，证明本软件为独立开发的原创作品。

1.2 软件概述

本软件是一款基于微信小游戏平台的休闲益智类水果主题小游戏。游戏以“从水果碗中捞取水果并完成订单”为核心体验，融合了订单目标、暂存区策略、道具救场、每日限定挑战、切水果无尽模式、

图鉴收集、扭蛋奖励、排行榜和云端同步等功能。玩家需要在限定的暂存空间和不断变化的水果池中做出选择，完成关卡订单并解锁更多水果内容。

本软件的主要功能包括:

- * 捞水果关卡玩法: 玩家从水果碗中点击食材，按顶部订单盘要求完成指定水果订单
- * 暂存区策略系统: 非目标食材进入底部菜碟，菜碟满时触发失败或复活流程
- * 道具系统: 加菜碟、移除、打乱等工具用于提高容错和改变局势
- * 每日限定玩法: 每日生成特定饮品主题目标，支持分享配方和额外奖励
- * 切水果无尽玩法: 通过水果网格合成、管道管理和里程碑奖励形成长线挑战
- * 图鉴系统: 记录水果解锁进度、展示食材图标和收集状态
- * 扭蛋系统: 使用金币抽取道具或奖励，形成局外资源循环
- * 排行榜系统: 展示玩家进度和挑战表现，支持世界榜与好友榜能力
- * 云同步系统: 支持本地进度与云端存档同步，保障跨设备数据一致
- * 数据分析系统: 上报玩法进入、关卡通关、失败、道具使用等事件，用于运营分析



图1 首页主界面 - 三种玩法入口与底部功能入口

1.3 运行环境

项目	要求
运行平台	微信小游戏
操作系统	iOS 10.0及以上 / Android 5.0及以上
微信版本	微信客户端 6.7.2 及以上
屏幕方向	竖屏(Portrait)
开发语言	TypeScript / JavaScript (ES6+)

渲染技术	PixiJS + WebGL/Canvas
构建工具	Vite
云服务	微信云开发 / HTTP 云函数
网络要求	核心关卡可本地运行，云同步、排行榜、数据上报需要网络连接

1.4 术语与缩略语

术语/缩略语	含义说明
订单	关卡中要求收集指定数量水果的目标单元
暂存区	底部菜碟区域，用于暂放无法立即交付订单的水果
订单盘	顶部用于显示当前订单目标和收集进度的盘子
复活	失败后通过广告或降级策略清空暂存区并继续本关
每日限定	按日期生成主题饮品目标的特殊挑战玩法
无尽模式	不固定关卡终点，以最高分和里程碑为目标的持续挑战玩法
PixiJS	用于小游戏渲染的 2D WebGL 渲染引擎
Scene	游戏场景，负责独立界面生命周期、渲染和输入处理
SDK	Software Development Kit，软件开发工具包
API	Application Programming Interface，应用程序接口

二、软件总体设计

2.1 软件需求概括

本软件采用模块化的软件设计方法，以微信小游戏框架为基础平台，使用 TypeScript 编写核心逻辑，通过 Vite 构建为小游戏可运行的 JavaScript 包。软件采用单页面应用架构，使用场景管理器切换首页、关卡、每日限定、切水果、图鉴、扭蛋、排行榜等功能界面。

- * 高性能实时渲染: 游戏包含大量水果精灵、飞行动画、弹窗和粒子效果，需要稳定帧率
- * 准确触摸输入: 支持点击水果、点击工具、弹窗按钮和排行榜滚动等多类交互
- * 可靠进度存储: 关卡进度、金币、图鉴、道具库存和最高分需要本地持久化并可云端同步
- * 可扩展内容配置: 水果、关卡、主题、徽章、经济和每日限定数据均采用配置化管理
- * 稳定网络降级: 广告、排行、云同步和数据分析失败时不影响核心玩法继续运行

2.2 总体架构设计

本软件采用"启动入口 + 引擎核心 + 场景系统 + 配置数据 + 服务模块 + 工具组件"的分层架构。入口层负责初始化画布、音频、分享、云同步和场景注册；引擎核心封装 PixiJS 应用、场景切换、平台适配和本地持久化；业务场景负责具体玩法和界面逻辑。

系统整体模块信息如下:

模块层	模块名称	功能简述
-----	------	------

入口层	game.js / src/main.ts	创建小游戏运行环境，初始化游戏核心并注册场景
核心层	core/Game.ts	封装 PixiJS 舞台、逻辑尺寸、Ticker 和安全区
核心层	core/SceneManager.ts	统一管理场景注册、预加载、切换和生命周期
场景层	scenes/HomeScene.ts	首页入口、金币栏、底部导航和玩法入口
场景层	scenes/BowlScene.ts	主关卡玩法，订单、碗、暂存区、道具和通关流程
场景层	scenes/DailyLimitedScene.ts	每日限定饮品挑战玩法
场景层	scenes/FruitSliceEndlessScene.ts	切水果无尽挑战玩法
配置层	config/*.ts	水果、关卡、主题、经济、云服务和素材路径配置
服务层	services/*.ts / managers/*.ts	排行榜、云同步、用户资料、隐私授权和数据上报
云函数	cloudfunctions/*	存档、鉴权、排行榜、数据接收和关卡通关率接口

2.3 模块划分与关系

软件主要依赖关系如下：

```
game.js
-> minigame/game-bundle.js
-> src/main.ts
    -> core/Game.ts
    -> core/SceneManager.ts
    -> managers/CloudSyncManager.ts
    -> scenes/HomeScene.ts
    -> scenes/BowlScene.ts
    -> scenes/DailyLimitedScene.ts
    -> scenes/FruitSliceEndlessScene.ts
    -> scenes/CatalogScene.ts / GachaScene.ts / LeaderboardScene.ts
    -> config/*.ts / game/*.ts / gameobjects/*.ts / services/*.ts
```

各场景通过 SceneManager 进行解耦，进入场景时执行 onEnter，离开场景时执行 onExit，需要提前准备资源的场景实现 prepare 或内部预加载流程。共享能力如音频、云同步、排行榜、纹理缓存和广告能力以服务模块形式提供，避免在场景之间直接耦合。

2.4 场景系统设计

场景	功能描述	关键模块
home	首页与玩法入口	HomeScene
bowl	主关卡玩法	BowlScene
dailyLimited	每日限定玩法	DailyLimitedScene
fruitSlice	切水果无尽玩法	FruitSliceEndlessScene
catalog	图鉴展示	CatalogScene
gacha	扭蛋抽奖	GachaScene
leaderboard	排行榜	LeaderboardScene

三、核心模块详细设计

3.1 游戏入口模块

游戏入口由 game.js 与 src/main.ts 协同完成。game.js 作为微信小游戏启动文件，加载构建后的 bundle；src/main.ts 获取 Canvas、初始化 Game、设置分享、初始化数据分析、等待云同步启动门、注册所有场景，并根据玩家是否已有进度决定进入第一关或首页。

```
Game.init(canvas)
setupWechatShare()
initAnalytics()
CloudSyncManager.prewarm()
SceneManager.register(homeScene)
SceneManager.register(bowlScene)
SceneManager.switchTo(shouldEnterFirstLevel ? "bowl" : "home")
```

3.2 场景管理模块

SceneManager 负责维护场景注册表和当前场景引用。切换场景时，先调用旧场景 onExit 清理定时器、原生按钮和临时节点，再调用新场景 onEnter 挂载界面，确保同一时间只有一个业务场景响应输入。

LoadingOverlay 用于进入较重场景时展示进度，TextureCache 负责缓存已加载的纹理，loadBowlSubpackage 等工具函数负责按需加载分包资源，降低首包压力。

3.3 捞水果关卡玩法模块

BowlScene 是主玩法核心模块，负责关卡定义读取、水果生成、订单分配、碗内点击、飞行动画、暂存区管理、失败复活、道具执行、通关奖励、徽章解锁和排行榜进度上报。



图2 关卡玩法界面 - 碗内食材、订单盘与暂存区

捞水果关卡核心流程如下:

```
startRound()  
-> 读取关卡配置并生成水果池  
-> 初始化订单盘、暂存槽和碗内水果  
-> 玩家点击水果  
    -> 若匹配订单则飞入订单盘并更新进度  
    -> 否则飞入暂存区  
-> 订单完成后补下一单并释放隐藏水果  
-> 全部订单完成后展示通关界面  
-> 暂存区满时展示失败复活界面
```

子模块	功能说明
订单系统	根据关卡水果池分配订单，维护每个订单盘的目标水果和完成进度
碗内水果系统	管理水果位置、层级、飞行动画、冻结状态和隐藏释放策略
暂存区系统	管理底部菜碟容量、占用状态、紧迫态提醒和复活清空
道具系统	提供加菜碟、移除、打乱三类救场工具
通关结算	展示解锁内容、近30天通关数据、下一关和分享入口
失败复活	提供广告复活、重玩本关和返回首页入口



图3 关卡通关界面 - 解锁内容与通关数据



图4 关卡失败复活界面 - 订单进度与复活操作

3.4 每日限定玩法模块

DailyLimitedScene 提供按日期变化的饮品主题挑战。关卡从 dailyLimitedLevels 配置读取当天目标水果、饮品名称、配方卡和主题资源，玩家需要在有限暂存空间内完成指定目标。该模式包含洗牌、撤

销、抬起等专用道具，并对开始、结束、分享、额外格子解锁和道具使用进行数据上报。



图5 每日限定界面 - 目标水果与暂存策略

3.5 切水果无尽玩法模块

FruitSliceEndlessScene 提供分数驱动的无尽挑战玩法。玩家通过水果网格和管道进行合成、清除和刷新，在管道或网格压力达到上限前尽可能获得更高分数。该模式包含 checkpoint 续玩、复活、里程碑奖励、金币结算和排行榜上报等机制。

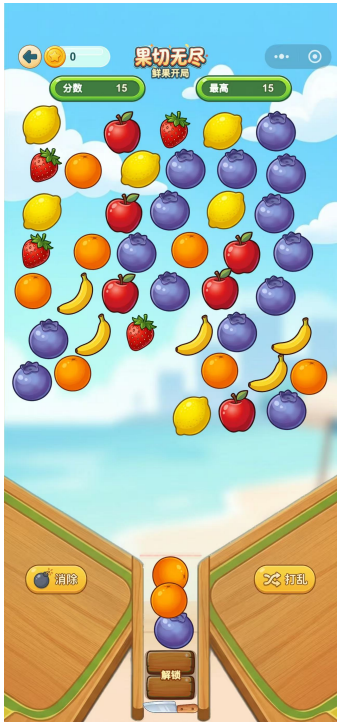


图6 切水果无尽界面 - 水果网格与管道区域

3.6 图鉴、扭蛋与排行榜模块

CatalogScene 负责展示食材图鉴和解锁状态；GachaScene 负责扭蛋抽奖、奖励发放和库存更新；LeaderboardScene 负责展示世界榜和好友榜，并与隐私授权、用户资料和开放数据域联动。



图7 图鉴界面 - 食材解锁进度



图8 扭蛋界面 - 奖励抽取与道具库存



图9 排行榜界面 - 玩家进度排名

3.7 云同步与数据上报模块

CloudSyncManager 负责启动阶段的权威数据拉取、运行中的防抖推送和后台 flush。BackendService 封装鉴权、存档、排行榜和关卡通关率接口；analytics 模块负责将用户行为事件批量上报到 analytics

-ingest 云函数。



图10 设置界面 - 音频与返回控制

四、数据结构设计

4.1 关卡配置数据结构

```
interface BowlLevelDef {
  levelNumber: number
  displayName: string
  fruitIds: FruitId[]
  copiesPerFruit: number
  orderTarget: number
  bufferSize: number
  iceCount?: number
  frozenCount?: number
  allowAddDish: boolean
  allowRemove: boolean
  allowShuffle: boolean
}
```

4.2 水果与订单状态数据结构

```
FruitItem: fruitId、phase、picked、frozen、bufferSlotIndex、position、velocity
parallelOrders[plateIdx]: { fruitId, progress }
bufferSlots[index]: FruitItem | null
remainingCounts[fruitId]: number
```

pendingOrderPlateCounts[plateIdx]: number

4.3 玩家进度与经济数据结构

BowlProgress: 当前关卡、已解锁徽章、图鉴解锁状态
Wallet: 金币余额、加减金币记录
ToolInventory: 加菜碟、移除、打乱等道具库存
FruitSliceProgress: 切水果最高分、checkpoint、奖励领取状态
PersistService: 本地 JSON 读写封装

4.4 云端数据结构

数据类型	关键字段	用途
用户存档	userId、snapshot、updatedAt	云端保存玩家进度
排行榜记录	board、score、rank、profile	展示关卡进度和无尽分数排名
通关率快照	level_id、pass_rate、start_users、clear_time	展示近30天关卡通关数据
分析事件	event_name、user_id、payload、created_at	统计玩法行为和转化数据

五、数据接口设计

5.1 本地存储接口

本地存储由 PlatformService 和 PersistService 统一封装，业务模块使用同步读写接口保存进度、设置、金币、道具、图鉴和切水果记录。存储失败时使用默认值兜底，不阻塞游戏流程。

5.2 云服务接口

接口	功能说明
/auth/login	获取用户身份令牌并绑定平台用户标识
/save/pull	拉取云端权威存档
/save/push	推送本地最新存档快照
/rank/submit	提交排行榜成绩
/rank/list	分页获取排行榜列表
/level/pass-rates	获取近30天关卡通关率快照
analytics-ingest	接收客户端批量行为事件

5.3 微信平台接口

本软件使用微信小游戏 Canvas、分包加载、云开发数据库、激励视频广告、分享、开放数据域和隐私授权等能力。所有平台调用均通过工具层封装，并在不可用或失败时静默降级。

六、出错处理设计

6.1 网络异常处理

- * 云同步失败时保留本地存档，并在下次启动或后台 flush 时重试
- * 排行榜拉取失败时显示空状态或缓存数据，不影响核心关卡
- * 关卡通关率接口失败时隐藏通关数据卡片或使用本地缓存
- * 广告不可用时根据业务场景提示稍后重试或走不可用降级逻辑

6.2 数据异常处理

- * 配置缺失时使用默认值兜底，防止关卡初始化中断
- * 存档 JSON 解析失败时返回默认结构，避免坏档导致白屏
- * 无效水果 ID 或纹理缺失时跳过绘制，并在控制台记录警告
- * 排行榜用户资料缺失时使用匿名展示，不阻塞成绩提交

6.3 运行时异常处理

- * 场景退出时清理定时器、ticker、原生按钮和临时容器，避免跨场景残留
- * 输入处理在动画、弹窗或教学锁定期间提前返回，避免重复触发
- * 资源加载失败时保留程序绘制兜底 UI，确保主要功能可用
- * 云函数异常由服务层捕获并返回业务可识别错误码

七、性能优化设计

7.1 资源管理优化

- * 通过微信分包拆分关卡、主题、徽章、图鉴、每日限定、切水果和扭蛋资源
- * TextureCache 统一缓存纹理，避免重复创建 Sprite 纹理对象
- * 首页和玩法入口只预加载关键资源，其他资源按场景进入时加载
- * 场景退出时销毁动态创建的 Pixi 容器和文本节点，减少内存泄漏

7.2 渲染性能优化

- * 使用 PixiJS 承担精灵批处理、纹理管理和舞台渲染
- * 将碗内水果、飞行层、汤底层、UI 层分层管理，减少层级错乱和重绘复杂度
- * 高频动画由 Game.ticker 统一驱动，完成后及时 remove
- * 按钮贴图和复杂 UI 采用预制素材，降低运行时矢量绘制成本

7.3 加载与包体优化

- * Vite 构建输出单一小游戏 bundle，便于微信小游戏加载
- * 图片资源按玩法拆分到 subpackages，降低首包体积
- * LoadingOverlay 在进入重场景时展示进度，避免用户感知卡顿

* 音频和广告预热采用后台异步方式执行，不阻塞首屏渲染